

C++ | CPl

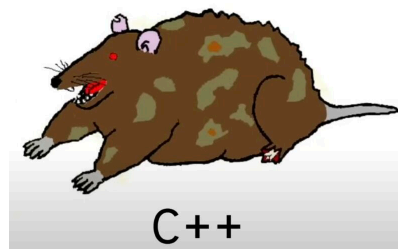
Summary

CONTENTS

1. Why C++?	2
2. Intro	2
2.1. Compilation	2
2.2. Declarations and definitions	2
2.2.1. #include guards	3
2.2.2. Values and references	3
2.3. Libraries	3
Bibliography	4

1. WHY C++?

- Headaches (if you're a masochist)
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Has a pretty mascot [\[1\]](#)



2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)
 - Nice and clean: `int main() { }` Insanity: `auto main() → int { }`
- Header files can be either `*.hpp` or `*.h`

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (One Definition Rule), in contrast to declaration, which can be done several times.

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.1. #include guards

#include guards ensure that a header file is only included once.

```
#ifndef GUARD_HPP_  
#define GUARD_HPP_  
// ...  
#endif /* GUARD_HPP_ */
```

2.2.2. Values and references

TODO:

C++ allocates memory for variables on definition

- No explicit heap memory needed
- No indirection and space overhead

2.3. LIBRARIES

- catch2
- boost

BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sept. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE